

RIPASSO GRAFI

Un **GRAFO** è una coppia $G = (V, E)$, con V un insieme di **NODI** ed E un insieme di **ARCHI** (orientati o bidirezionali)

- Due nodi sono **ADIACENTI** se sono collegati da un arco
- Un **CAMMINO** tra due nodi v_1 e v_2 è un insieme di archi di cui il primo ha origine in v_1 , l'ultimo termina in v_2 e ogni altro nodo compare almeno una volta sia come destinazione di un arco che come sorgente
- Un grafo è detto **ORIENTATO**, se la coppia di nodi che costituisce un arco è ordinata
- Grafo **CONNESSO**: esiste un cammino per ogni coppia di nodi
- Grafo **COMPLETO**: esiste un arco per ogni coppia di nodi
- **CICLO**: cammino dove il nodo iniziale è uguale a quello finale
- Grafo **ACICLICO**: Grafo privo di cicli

Un grafo si può rappresentare con:

- Liste di adiacenza $\leftarrow$ conviene quando ho pochi archi
 - Comp. spaziale: $\Theta(|V| + |E|)$
 - Ricerca di un arco: $O(|V|)$
- Matrice di adiacenza $\leftarrow$ conviene quando il grafo è più denso
 - Comp. spaziale: $\Theta(|V|^2)$
 - Ricerca di un arco: $\Theta(1)$

OPERAZIONI POSSIBILI:

• visita in ampiezza

- Ordine di visita: vengono visitati tutti i nodi con un cammino tra loro e la sorgente s lungo n passi, prima di visitare quelli con un cammino lungo $n + 1$

VISITAAMPIEZZA(G, s)

```
1 for each  $n \in V \setminus \{s\}$ 
2    $n.color \leftarrow white$ 
3    $n.dist \leftarrow \infty$ 
4  $s.color \leftarrow grey$ 
5  $s.dist \leftarrow 0$ 
6  $Q \leftarrow \emptyset$  ← uso una coda
7 ENQUEUE( $Q, s$ )
8 while  $\neg ISEMPTY(Q)$ 
9    $curr \leftarrow DEQUEUE(Q)$ 
10  for each  $v \in curr.adiacenti$ 
11    if  $v.color = white$ 
12       $v.color \leftarrow grey$ 
13       $v.dist \leftarrow curr.dist + 1$ 
14      ENQUEUE( $Q, v$ )
15   $curr.color \leftarrow black$ 
```

- Linee 1-7 : inizializzano tutti i nodi come bianchi
- Linee 8-15: effettuano la visita del grafo
- N.B. Ogni arco è visitato una sola volta (a partire dal nodo sorgente)
- Complessità totale: $O(|V| + |E|)$

• visita in profondità

- Ordine di visita: segue i cammini "fino in fondo" prima di visitare i vicini del nodo di partenza

VISITAPROFONDITÀ(G, s) (versione iterativa)

```
1 for each  $n \in V \setminus \{s\}$ 
2    $n.color \leftarrow white$ 
3    $n.dist \leftarrow \infty$ 
4  $s.color \leftarrow grey$ 
5  $s.dist \leftarrow 0$ 
6  $P \leftarrow \emptyset$  ← uso una PILA
7 PUSH( $P, s$ )
8 while  $\neg ISEMPTY(P)$ 
9    $curr \leftarrow POP(P)$ 
10  for each  $v \in curr.adiacenti$ 
11    if  $v.color = white$ 
12       $v.color \leftarrow grey$ 
13       $v.dist \leftarrow curr.dist + 1$ 
14      PUSH( $P, v$ )
15   $curr.color \leftarrow black$ 
```

- Linee 1-7 : inizializzano tutti i nodi come bianchi
- Linee 8-15: effettuano la visita del grafo
- N.B. Ogni arco è visitato una sola volta (a partire dal nodo sorgente)
- Complessità totale: $O(|V| + |E|)$

- Ordine di visita: segue i cammini "fino in fondo" prima di visitare i vicini del nodo di partenza

(versione ricorsiva)

VISITAPROFONDITÀ(G)

```
1 for each  $u \in V$ 
2    $u.color \leftarrow white$ 
3 for each  $u \in V$ 
4   if  $u.color = white$ 
5     DFSRICORSIVA( $G, u$ )
```

DFSRICORSIVA(G, u)

```
1  $u.color \leftarrow grey$ 
2 for each  $v \in u.adiacenti$ 
3   if  $v.color = white$ 
4     DFSRICORSIVA( $G, v$ )
5  $u.color \leftarrow black$ 
```

- Complessità totale: $O(|V| + |E|)$

OSSERVA: se il grafo fosse uguale a un albero, la visita in profondità sarebbe uguale a una visita in post-order

• Trovare le COMPONENTI CONNESSE

- É detta *componente connessa* di un grafo G un insieme S di nodi tali per cui esiste un cammino tra ogni coppia di essi, ma nessuno di essi è connesso a nodi $\notin S$

COMPONENTI CONNESSE(G)

```
1 for each  $v \in V$ 
2    $v.etichetta \leftarrow -1$ 
3  $eti \leftarrow 1$ 
4 for each  $v \in V$ 
5   if  $v.etichetta = -1$ 
6     VISITA E ETICHETTA( $G, v, eti$ )
7      $eti \leftarrow eti + 1$ 
```

- VISITA E ETICHETTA funziona come VISITA AMPIEZZA o VISITA PROFONDITÀ ma imposta a eti il campo *etichetta* del nodo visitato
- Complessità: $\mathcal{O}(|V| + |E|)$, ogni nodo viene visitato una sola volta

• Trovare l'ordinamento topologico

- Dato un grafo orientato aciclico, il *predecessore* di un nodo v è un nodo u tale per cui esiste un cammino da u a v
- L'ordinamento topologico è una sequenza di nodi del grafo tale per cui nessun nodo compare prima di un suo predecessore (non è unico!)

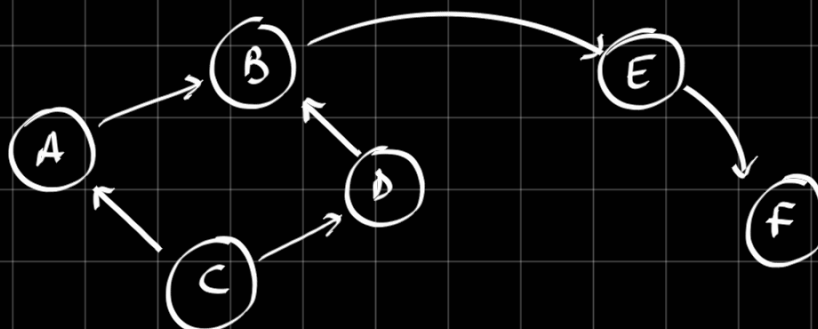
ORDINAMENTO TOPOLOGICO(G)

```
1  $L \leftarrow \emptyset$ 
2 for each  $v \in V$ 
3    $v.color \leftarrow white$ 
4 for each  $v \in V$ 
5   if  $v.color = white$ 
6     VISITA PROFOT( $G, v, L$ )
7 return  $L$ 
```

VISITA PROFOT(G, s, L)

```
1  $s.color \leftarrow grey$ 
2 for each  $v \in s.adiacenti$ 
3   if  $v.color = white$ 
4     VISITA PROFOT( $G, v, L$ )
5  $s.color \leftarrow black$ 
6 PUSHFRONT( $L, s$ )
```

ESEMPIO :



- Non è detto che l'algoritmo parta da C. Per esempio, iniziamo dal nodo D
- L'unico nodo adiacente è B $\rightarrow$ Mi sposto lì.
- L'unico nodo adiacente è E $\rightarrow$ Mi sposto lì.

- L'unico modo adiacente è $F \rightarrow$ Mi sposto lì.
- Non ho modi adiacenti a $F \rightarrow$ Coloro F di nero e lo riporto in fondo all'array $ARR = [\dots \dots F]$
- Torno indietro $\rightarrow$ faccio lo stesso con E, B e D . $ARR = [\dots \dots D \ B \ E \ F]$
- Scelgo un nuovo modo a caso: questa volta prendo C
- Un suo modo adiacente è A
- Non ho modi bianchi adiacenti a $A \rightarrow$ Coloro A di nero, lo metto in ARR e torno indietro.
 $ARR = [C \ A \ D \ B \ E \ F]$

L'algoritmo funziona: per ogni modo nell'array, i suoi predecessori stanno prima

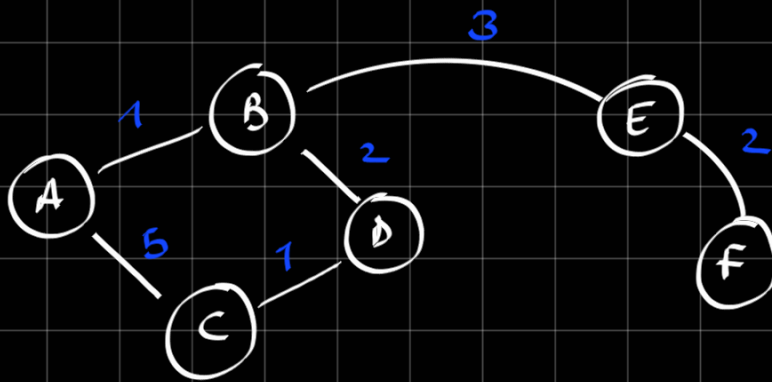
L'ordine topologico non è unico: in questo esempio si potrebbero scambiare A e D

• Percorso più breve: **ALGORITMO DI DIJKSTRA**

- Trova, dato un grafo orientato e un suo nodo s , i percorsi più brevi da s a ogni altro nodo
 - Funziona sia su di un grafo classico, che su di un grafo *pesato* ovvero con archi dotati di un valore intero *positivo* (cammini a somma dei pesi minima)
- Principio di funzionamento
 - Inserisco ogni $v \in V \setminus \{s\}$ in un insieme Q dopo aver impostato il suo attributo distanza a ∞ ed il $v.pred$ a NIL
 - Inserisco s in Q dopo aver impostato $s.dist \leftarrow 0, s.pred \leftarrow NIL$
 - Fin quando Q non è vuoto, estraggo il nodo c con $dist$ minima e controllo per ogni adiacente a se hanno distanza minore di $c.dist + peso(c, a)$
 - Se questo accade imposto $a.pred \leftarrow c, a.dist \leftarrow c.dist + peso(c, a)$
- Complessità temporale: $O(|V| \log(|V|) + |E|)$

VANTAGGIO: Ottengo tutti i costi per raggiungere qualsiasi modo dal nodo sorgente.

ESEMPIO DI PRIMA:



partiamo, in questo caso, dal nodo A.

A	B	C	D	E	F	
0	∞	∞	∞	∞	∞	[]

• Guardo i nodi adiacenti ...

A	B	C	D	E	F	
0	1	5	∞	∞	∞	[B C]

ripeto con il nodo non escluso e con costo minimo, cioè B

• Guardo i nodi adiacenti ...

A	B	C	D	E	F	
0	1	5	3	4	∞	[C D E]
		↑	↑			
		2+1	3+1			

Adesso scelgo D

• Guardo i nodi adiacenti ...

A	B	C	D	E	F	
0	1	4	3	4	∞	[C E]

Posso raggiungere C più velocemente facendo il percorso A-B-D-C $\rightarrow$ Aggiorniamo il costo

Possiamo scegliere C o E come prossimo nodo; facciamo con C...

• Guardo i nodi adiacenti...

A	B	C	D	E	F	
0	1	4	3	4	∞	[E]

• Guardo i nodi adiacenti...

A	B	C	D	E	F	
0	1	4	3	4	6	[F]

• Guardo i nodi adiacenti...

A	B	C	D	E	F	
0	1	4	3	4	6	[]

SPIEGAZIONE COMPLESSITÀ:

$|V|$ perché visito tutti i nodi. Per ognuno di essi cerco il nodo con distanza minima. Sappiamo che, usando un HEAP, la ricerca del minimo costa $O(\log|V|)$

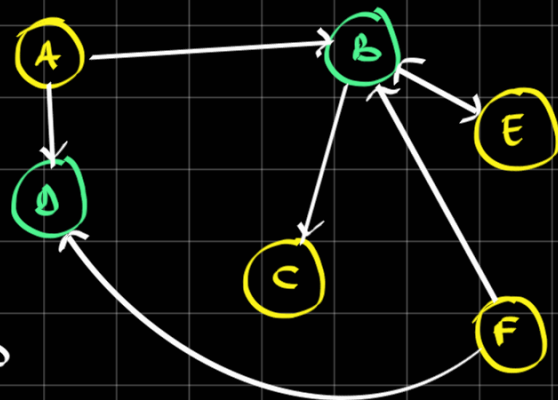
NOTA: l'algoritmo di Dijkstra funziona solo quando i pesi sono tutti positivi

1

Un grafo (connesso, non orientato) $\mathcal{G} = (V, E)$ è detto bipartito se V può essere partizionato in due sottoinsiemi X, Y tali che ogni arco in E collega un nodo $x \in X$ ed un nodo $y \in Y$. Si descriva un algoritmo che prende in input un grafo $\mathcal{G}$ e ritorna true se il grafo è bipartito, false altrimenti, e se ne valuti la complessità temporale.

SOLUZIONE:

Un grafo bipartito può essere il seguente



Un approccio per risolvere questo problema è fare una visita in ampiezza: per ogni nodo, indico il suo colore (facciamo nero o rosso) e vedo se i suoi nodi adiacenti sono del colore opposto. Se questa proprietà non vale, allora return false

Essendo letteralmente una visita in ampiezza del grafo, la complessità di questo algoritmo sarà $O(|V| + |E|)$



Non è detto che li visitiamo tutti: ci fermiamo prima se la proprietà non vale

2

Si consideri il seguente problema: dati in ingresso un automa a stati finiti e uno dei suoi stati finali, sia esso q , si deve dire se q è raggiungibile dallo stato iniziale. Si descriva un algoritmo per il problema e se ne valuti la complessità temporale.

SOLUZIONE:

Effettivamente, un ASF è facile da rappresentare come grafo.

Anche qui, quindi, basta fare una visita in ampiezza partendo dal nodo sorgente, e ritornare true se, prima di visitare tutti gli stati, ho raggiunto q .

Anche qui, poiché stiamo visitando tutti i nodi al più una volta, la complessità principale è $O(|V| + |E|)$